Lab01 JFlex Group Project

Compilation I4 GIC | Academic year 2026-2027 | Lecturer OL Phearun

Work in a group of 3 or 4 students to propose, build and test a small lexical analyzer using JFlex and Java. Choose a text format or a small language, define its tokens, and show how your scanner recognizes them. Finish the project within one week and present your work in 10 minutes, including a live demonstration.

Your project idea

Propose your own idea or adapt one from the idea bank. Keep the input format small enough to explain precisely. A console program that reads a text file is sufficient. Full parsing, expression evaluation, a graphical interface, a database and online services are outside the required scope.

Before coding, submit a short proposal with the project title, all members and student IDs, purpose, input format, planned token types, one sample input and expected output, and each member's responsibilities. State one feature you will leave outside the scope.

Minimum requirements

- Write a JFlex specification with named regular expression macros and Java actions. Generate the scanner with JFlex and compile and run it with Java.
- Define at least 6 useful token types. Whitespace, skipped comments and error messages do not count toward the 6. State which characters and formats your language allows.
- Read a UTF-8 text file. For each token, show its type, matched lexeme, line and column. Display positions starting at 1.
- Document whitespace and comment handling. Report unexpected characters with their positions and state whether scanning continues or stops. Handle end of input cleanly.
- Explain 2 cases where rules could compete. Show how the scanner resolves them, using examples such as a keyword versus a longer identifier or a long operator versus a short one.
- Draw one small NFA or DFA for one token pattern and explain its link to your regular expression. Use at least 15 recorded tests and include a test you created after finding a bug.

Example of token output

For the input total=12; on the first line, a scanner might produce the following. Your group chooses its own token names and output format.

Token type	Lexeme	Line	Column
IDENTIFIER	total	1	1
ASSIGN	=	1	6
INTEGER	12	1	7
SEMICOLON	;	1	9

Working as a group

Share specification design, implementation, testing and presentation work. With 3 members, combine documentation and test coordination. With 4 members, split those tasks. Every member must explain at least one rule, show their contribution and take part in the presentation. Review each other's work before submission.

Project ideas

Choose one idea if your group needs a starting point. Adapt the vocabulary, examples and limits to your own proposal. Each option can meet the same requirements without a large application.

1 Configuration file scanner

Recognize keys, assignment signs, integer values, decimal values, quoted strings, Boolean words and section brackets. Input: retries=3 Also test: enabled=true and a missing closing quote. Keep arrays and nested sections outside the scope.

2 Task command scanner

Recognize command words, task names or IDs, integers, quoted descriptions, comparison operators and punctuation. Input: add task_7 "Read chapter 4"; Also test: add versus added and an unexpected @. The scanner lists tokens and does not execute tasks.

3 Robot command scanner

Recognize movement words, identifiers, integers, decimal values, assignment signs and delimiters. Input: move(10, 2.5); Also test: move versus mover and an unsupported symbol. Limit the project to token recognition without hardware.

4 Arithmetic input scanner

Recognize identifiers, integers, decimals, assignment signs, arithmetic operators and parentheses or semicolons. Input: total=12.5+qty*2; Also test: >= versus > if supported, and an unexpected character. Tokenize expressions without calculating results.

5 Log format scanner

Recognize a defined timestamp shape, log levels, field names, assignment signs, integers, quoted messages and brackets. Input: [INFO] code=200 message="Ready" Also test: INFO versus INFORMATION and an unfinished message. Use a small declared log format.

6 File metadata scanner

Recognize reserved field names, identifiers, quoted paths, integers, assignment signs, size units and separators. Input: file="report.pdf"; size=2048; Also test: file versus filename and a missing quote. Recognize text without opening files or validating the operating system.

Scope choices

Define a teaching subset for formats such as paths, URLs, dates or logs. List the accepted forms and exclusions. For example, a timestamp token may match a date shape without checking the calendar. A sequence of valid tokens may still form an invalid statement. Explain which checks your project actually performs.

Testing and submission

A manageable week of work

Agree on the scope and expected output first. Then build a small scanner that runs, add the remaining rules, test competing patterns and errors, and prepare the demonstration. Organize work around your group's availability. Complete and rehearse the project within one week. Follow the submission and presentation dates announced by the lecturer.

Evidence from tests

Record at least 15 distinct tests: 6 normal inputs, 5 boundary or competing-rule cases, and 4 invalid inputs. Count each test once. Include empty input, adjacent tokens and end of input among the boundary cases. For invalid tests, use a character or malformed construct that your written lexical rules reject. Do not label a valid token sequence as a lexical error just because its statement syntax is wrong.

Test ID and kind	Input	Expected output	Actual result
N01 normal	Your input	Token sequence and positions	Observed output and pass or fail
B01 boundary	Your competing or edge case	Expected rule and reason	Observed output and pass or fail
E01 invalid	Your invalid input	Error and position	Observed output and pass or fail

Write expected results before running the tests. Keep the test inputs and expected output as text files. Show the corrected test for at least one bug. If a feature remains incomplete, report the limitation honestly.

One submission per group

Submit one ZIP named Lab01_GroupID_ProjectName.zip. Include all of the following:

- Source: your .jflex specification and any Java driver or support classes. Keep the generated scanner identifiable as generated code.
- Tests: input files, expected output and a table of all 15 or more results. Include the files used in the live demo.
- README: member names and IDs, JDK and JFlex versions, exact generation, compilation and run instructions, and known limitations.
- Report: 2 to 3 pages covering scope, token definitions, 2 competing-rule cases, one automaton, a bug and its correction, and a contribution record.
- Slides: the final presentation deck and a PDF copy. Keep backup screenshots of the demo in case the live run fails.

Understanding your scanner

Use lecture notes and the official JFlex manual. Write and test the specification as a group. Do not submit an AI-generated or downloaded solution as your own work. Cite reused examples and assistance. Every member should be ready to explain or adapt one rule when the lecturer asks.

Technical reference: JFlex User's Manual, https://jflex.de/manual.html. JFlex chooses the longest match, then the earliest rule when matches have equal length. The %line and %column options enable zero-based counters, so add 1 when displaying human-readable positions.

Presentation and demonstration

Prepare about 7 slides and rehearse the whole presentation within 10 minutes, including switching to the program and running the demo. Use concise slides and explain the details aloud. The editable template follows this outline. Replace its prompts with evidence from your own project.

Slide	What your group should show
1	Project and team. Give the project title, member names and IDs, and the purpose in one sentence.
2	Input and scope. Show one short input example, expected output and clear limits.
3	Token definitions. Use a compact table of token types, patterns and examples. Explain 2 competing-rule cases.
4	Scanner design. Show the path from input to scanner output and one small token automaton. Explain where your JFlex rules fit.
5	Test evidence. Show the number of tests in each category, actual results, and one bug before and after correction.
6	Live demonstration. Run a normal input, a competing-rule or boundary input and an invalid input. Point out token types and positions.
7	Results and contributions. Explain what works, one limitation and each member's contribution. End with the main lesson from the project.

Demo checklist

- Bring the .jflex file, Java source, test files and a working Java and JFlex setup. Keep the project runnable without internet access.
- Show the generation and compilation commands, then run the prepared inputs. Make the console text large enough for the class to read.
- For each input, state the expected behavior before running it. Explain the actual output, including error locations.
- Be ready to scan one small input chosen by the lecturer. If the live demo fails, use the saved output and explain the issue.

What the lecturer will review

The review considers the clarity of the token specification, correct use of JFlex, handling of competing rules and errors, quality of test evidence, reproducible execution, and each member's understanding and contribution. Follow any grading weights announced separately by the lecturer.

Final check before submission

A teammate should follow the README in a clean folder, regenerate and compile the scanner, run the tests and compare the output. Check that every file opens, the slides match the submitted program, and all members can explain the work.